

mabl para QA Low-Code com IA

Guia prático do aluno com CopaFigurinhas

Browser tests manuais • Assertions HTML/CSS • Visual assertions • Build with mabl

1

TESTE MANUAL

2

VALIDAÇÃO VISUAL

3

IA NA AUTORIA

Fluxo prioritário pelo navegador e execução em cloud

Atualizado em 17/08/2026 com base no fluxo validado em aula e na documentação oficial atual do mabl.

FOCO DESTES GUIA

Você vai configurar Application, Environment e Base URL, criar testes manuais no Trainer, validar HTML/CSS, usar visual assertions e criar um teste com IA pelo fluxo Build with mabl. Não vamos depender de Visual Test standalone, Desktop App, CLI, Plans ou DataTables.

Como usar este guia

Este material foi escrito diretamente para você executar o laboratório. A proposta é aprender primeiro o que o teste faz e somente depois usar IA para acelerar a autoria. O objetivo não é conhecer toda a plataforma, e sim dominar um fluxo pequeno, confiável e reproduzível.

IMPORTANTE SOBRE VISUAL TEST

Na interface usada neste laboratório, a opção Visual Test standalone não estava disponível no menu de criação. Isso não impede o laboratório. Você fará validação visual dentro de Browser tests usando Visual assertions, recurso disponível no Trainer. Portanto, não procure Visual Test para concluir as atividades deste guia.

Etapa	O que você fará	Resultado
1. Preparação	Configurar Application, Environment e Base URL	mabl sabe qual aplicação e ambiente testar.
2. Manual	Criar Browser test no Trainer	Passos e assertions HTML/CSS ficam explícitos.
3. Visual	Adicionar Visual assertion	IA avalia critérios visuais objetivos.
4. Build with mabl	Criar teste por prompt	O agente propõe outline e constrói o teste em cloud.
5. Execução	Executar Cloud run e analisar evidências	Você interpreta PASS, FAIL e diagnóstico.

O que fica fora do escopo

- Visual Test standalone como requisito do laboratório.
- Desktop App e mabl Link como caminho principal.
- CLI, CI Runner, Plans, DataTables e auto-heal avançado.
- Testes destrutivos ou alterações simultâneas em dados compartilhados.

1. Preparação do ambiente

OBJETIVO

Chegar ao primeiro teste com um workspace ativo e uma URL pública estável do CopaFigurinhas. O fluxo cloud do mabl precisa alcançar a aplicação pela Internet.

1.1 Antes de abrir o mabl

- Tenha acesso ao workspace mabl definido para a turma.
- Tenha a URL pública do CopaFigurinhas fornecida pelo professor.
- Abra a URL em uma janela anônima e confirme que a aplicação carrega sem VPN ou acesso à sua máquina.
- Evite localhost neste guia. O cloud authoring e o Cloud run precisam alcançar a aplicação externamente.

1.2 Configurar Application, Environment e Base URL

1. Acesse o mabl Web App e confirme que está no workspace correto.
2. No menu principal, abra Configuration > Applications.
3. Clique em + Create application.
4. Use Application name: CopaFigurinhas.
5. Crie ou associe o Environment: Lab.
6. No Application target, informe a Base URL pública fornecida pelo professor: {URL_PUBLICA_COPAFIGURINHAS}.
7. Clique em Save.

Configuração	Valor do laboratório
Application	CopaFigurinhas
Environment	Lab

Application target / Base URL	{URL_PUBLICA_COPAFIGURINHAS}
Tipo	Web application URL

COMO O MABL ENXERGA ISSO

A Base URL é um application target associado ao par Application + Environment. Em Browser tests, essa URL é exposta como app.url e é o ponto inicial do teste.

1.3 Critério de prontidão

- Application CopaFigurinhas aparece em Configuration > Applications.
- Environment Lab está associado à aplicação.
- A Base URL abre normalmente fora da sua sessão local.
- Você consegue iniciar a criação de um Browser test.

2. Primeiro Browser test manual: smoke com HTML/CSS

OBJETIVO DIDÁTICO

Criar um smoke test simples sem IA. Você verá claramente a relação entre interação, passo e assertion.

2.1 Criar o teste

1. Clique em New test > Browser test.
2. Na tela de criação, escolha Create manually.
3. Nomeie o teste como CF | Smoke | HTML.
4. Em Core settings, selecione a Application CopaFigurinhas e o Environment Lab. Confirme a URL de treinamento.
5. Mantenha Desktop web para este laboratório.
6. Clique em Create test para abrir o mabl Trainer.

2.2 Entender o Trainer

- Record ligado: cliques, preenchimentos e navegação são gravados como passos.
- Ícone de check: adiciona assertions.
- Botão +: adiciona passos extras, como waits, viewport e outros recursos.
- Play: reproduz os passos.
- Save: salva a versão atual do teste.

REGRA DE OURO

Clique não é validação. Um teste só confirma comportamento quando possui assertions que representam o resultado esperado.

2.3 Adicionar assertions HTML/CSS

No Trainer, clique no ícone de check e escolha Element. Para este primeiro teste, use a opção de propriedades HTML/CSS, não Visual.

Alvo	Assertion sugerida	O que prova
Título CopaFigurinhas	Texto / innerText contém "CopaFigurinhas"	A página principal carregou.
Botão Compartilhar relatório	Texto contém "Compartilhar relatório"	A ação principal do cabeçalho está presente.
Campo Buscar jogador	placeholder contém "Nome do jogador"	A região de busca foi carregada.
Seção Adicionar figurinha	Texto contém "Adicionar figurinha"	O formulário principal está disponível.

2.4 Executar

1. Salve o teste.
2. Feche o Trainer ou volte à página de detalhes.
3. Clique em Run test.
4. Selecione Cloud run.
5. Clique em Start run.
6. Abra o resultado e confirme que os passos e assertions ficaram em PASS.

3. Teste manual funcional: buscar um jogador

OBJETIVO DIDÁTICO

Sair do smoke e validar uma interação real. Você executa uma ação e confirma o estado resultante da interface.

3.1 Cenário

Passo	Ação	Validação
1	Abrir a página principal	Tela carregada.
2	Preencher Buscar jogador com Marcos Vieira	Campo contém o valor.
3	Aguardar a filtragem	O card esperado permanece visível.
4	Criar Element assertion HTML/CSS	O card contém Marcos Vieira.
5	Criar segunda assertion	O card contém Brasil.

3.2 Como gravar

1. Crie um novo Browser test manual chamado **CF | Funcional | Buscar jogador**.
2. Com **Record** ligado, clique no campo Buscar jogador e digite **Marcos Vieira**.
3. Pare a gravação se não precisar registrar novas interações.
4. Adicione uma **Element** assertion **HTML/CSS** sobre o card filtrado.
5. Valide o nome **Marcos Vieira** e o país **Brasil**.
6. Salve e execute em Cloud run.

O QUE OBSERVAR NO RESULTADO

Se o teste falhar, primeiro verifique o screenshot do passo. Depois confirme se a falha está no produto, no teste, no dado ou no ambiente. FAIL não significa automaticamente bug do produto.

INTERAÇÃO → ASSERTION → EXECUÇÃO → EVIDÊNCIA

4. Visual assertions: validação visual com IA

Visual assertion não é um Visual Test standalone. Ela é uma assertion criada dentro do Browser test. O mabl avalia um screenshot da tela ou de um elemento com base em critérios gerados a partir do seu prompt.

4.1 Criar uma Visual assertion

1. Abra um Browser test no Trainer e coloque a aplicação no estado que deseja validar.
2. Clique no ícone de check para criar uma assertion.
3. Escolha Screen para avaliar o viewport inteiro ou Element para uma região específica.
4. Se escolher Element, selecione o elemento e escolha Visual.
5. Escreva um critério objetivo em linguagem natural.
6. Clique em Test assertion.

7. Revise os critérios gerados pelo mabl. Ajuste o prompt ou os critérios quando necessário.

8. Defina o comportamento em caso de falha e confirme.

PREFERÊNCIA PARA A AULA

Use Element visual assertion sempre que possível. Quanto menor e mais específico o alvo, menor a chance de uma validação visual ficar instável.

4.2 Prompts prontos

Prompt A: cards de resumo

Verifique se os cards de resumo estão legíveis, sem sobreposição de texto, com títulos e valores visíveis e alinhamento consistente. Ignore diferenças de conteúdo numérico que não afetem a legibilidade. Retorne a avaliação em português.

Prompt B: card de jogador

Verifique se este card de jogador apresenta nome, país, posição, quantidade e status de forma legível, sem texto cortado, elementos sobrepostos ou controles fora do card. Retorne a avaliação em português.

Prompt C: cabeçalho

Verifique se o botão "Compartilhar relatório" está totalmente visível dentro do cabeçalho e não se sobrepõe ao título nem ao subtítulo. Considere apenas posição, legibilidade e sobreposição. Retorne a avaliação em português.

4.3 Como evitar flakiness

- Mantenha o mesmo viewport entre criação e execução.
- Prefira critérios observáveis: legível, visível, alinhado, sem sobreposição.
- Evite frases subjetivas como "a página está bonita".
- Não tente validar conteúdo que está fora do viewport.
- Se a tela inteira variar muito, reduza o alvo para um elemento.

5. Build with mabl: criar teste com IA

O QUE É

Neste guia, Build with mabl é o fluxo de **autoria agentic** de **Browser tests**. Você descreve o objetivo, revisa o outline e deixa o agente construir os passos. A IA acelera a autoria, mas você continua responsável por revisar o teste.

5.1 Fluxo atual

1. Clique em New test > Browser test.
2. Em vez de **Create manually**, descreva o teste no campo de prompt do agente.
3. Envie o prompt e aguarde o mabl gerar o **Test outline**.
4. Revise as **tarefas**, **validações** e **Test information**. Corrija o outline pelo chat antes de continuar.
5. Clique em **Next**.
6. Confirme **Application** CopaFigurinhas, **Environment** Lab e a **URL** correta.
7. Em **Build approach**, selecione **Cloud**.
8. Clique em **Create test**.
9. Acompanhe a sessão de autoria em cloud até o teste ser concluído.
10. Abra o teste gerado, revise os passos e execute um Cloud run.

CORREÇÃO IMPORTANTE EM RELAÇÃO AO MATERIAL ANTIGO

Materiais anteriores podem mencionar o build approach "Generate". Na documentação atual, o fluxo agentic usa Build approach = Cloud ou Local. Para esta aula Web-first, use Cloud.

5.2 Prompts prontos para Build with mabl

Prompt 1: smoke

Valide a página principal do CopaFigurinhas. Confirme que o título "CopaFigurinhas", o botão "Compartilhar relatório", o campo "Buscar jogador" e a seção "Adicionar figurinha" estão disponíveis. Não altere dados da aplicação. Inclua assertions ao final das verificações principais.

Prompt 2: busca

Na página principal do CopaFigurinhas, pesquise por Marcos Vieira usando o campo "Buscar jogador". Confirme que o card de Marcos Vieira permanece visível e que o país Brasil aparece no card. Não clique nos botões +, - ou X e não altere outros jogadores.

Prompt 3: transição de estado

Pesquise por Rafael Andrade. Confirme que o card inicia com status FALTANTE e quantidade 0. Clique no botão + desse jogador. Confirme que a quantidade passa para 1 e que o status muda para OBTIDA. Não altere outros jogadores.

5.3 Prompts de correção do outline

Se o agente adicionar passos desnecessários

Simplifique o teste. Mantenha apenas os passos necessários para cumprir o objetivo e as assertions diretamente relacionadas ao resultado esperado. Não adicione navegação, cadastros ou alterações em outros jogadores.

Se o agente escolher o alvo errado

Use a Application CopaFigurinhas, o Environment Lab e a URL associada a esse ambiente. Não use outra aplicação, outro ambiente ou localhost.

6. Executar e interpretar os resultados

6.1 Cloud run

1. Abra a página de detalhes do teste.
2. Clique em Run test.
3. Selecione Cloud run.
4. Confirme Application, Environment e navegador.
5. Clique em Start run.
6. Abra a execução criada para acompanhar o resultado.

6.2 O que analisar

Evidência	Pergunta que responde
Status PASS / FAIL	O teste atingiu as expectativas definidas?
Screenshot	Como a interface estava naquele passo?
mabl Activity	Quais ações o mabl executou?
DOM	O elemento existia e com quais propriedades?
Network	Houve falha de chamada ou carregamento?
Console logs	O navegador registrou erro JavaScript?

Assertion output	Qual expectativa falhou?
------------------	--------------------------

6.3 Classifique a falha antes de corrigir

Categoria	Exemplo	Ação
Produto	Quantidade não atualizou	Corrigir a aplicação.
Teste	Assertion ficou rígida ou desatualizada	Corrigir o teste.
Dado	Jogador esperado não existe no estado atual	Restaurar ou adequar o dado.
Ambiente	Base URL indisponível	Restaurar o ambiente e repetir.

PERGUNTA CENTRAL

Quando um teste falha, não pergunte apenas “como fazer passar?”. Pergunte “o que essa falha está evidenciando e qual camada é responsável?”.

7. Desafio prático

Agora reproduza os três mecanismos principais sem copiar exatamente os exemplos anteriores.

Desafio	O que fazer	Evidência mínima
A. Manual HTML	Crie um Browser test manual para outro elemento da página e use pelo menos duas Element assertions HTML/CSS.	Teste salvo + Cloud run PASS.
B. Visual assertion	Escolha um card ou região diferente e escreva seu próprio prompt visual objetivo.	Prompt + critérios revisados + resultado.
C. Build with mabl	Crie um novo teste pelo agente para uma busca diferente ou outra interação não destrutiva. Revise o outline antes de Cloud build.	Prompt + outline revisado + Cloud run.

CRITÉRIO DE DOMÍNIO

Você concluiu quando consegue explicar: qual era o objetivo, quais passos foram executados, qual assertion confirmou o resultado e qual evidência sustenta PASS ou FAIL.

8. Troubleshooting rápido

Problema	Diagnóstico provável	O que fazer
Não encontro Visual Test	A opção standalone não está disponível no seu workspace/menu atual.	Continue com Browser test + Visual assertion. É o fluxo obrigatório deste guia.
Cloud authoring não abre a aplicação	Base URL não é pública, está errada ou exige acesso privado.	Revise Configuration > Applications e o Application target do Environment Lab.
Trainer abre a URL errada	Teste foi associado a outro Application/Environment/target.	Revise Core settings e app.url.
Assertion HTML pega elemento errado	Alvo muito genérico ou elementos semelhantes.	Selecione novamente o elemento e reduza o escopo.
Visual assertion oscila	Tela dinâmica, viewport diferente ou prompt subjetivo.	Use elemento específico, mesmo viewport e critério objetivo.
Build with mabl cria passos extras	Prompt amplo ou outline não revisado.	Refine o outline antes de Create test. Use um objetivo por teste.
Cloud run falha e Trainer passa	Diferença de ambiente, rede, dado ou timing.	Abra screenshot, Network, DOM e mabl Activity antes de alterar o teste.

9. Checklist final do aluno

☐ Application CopaFigurinhas configurada.

- ☐ Environment Lab configurado.
- ☐ Base URL pública associada ao Application target.
- ☐ CF | Smoke | HTML criado manualmente.
- ☐ Pelo menos três assertions HTML/CSS validadas.
- ☐ Teste funcional de busca executado em Cloud run.
- ☐ Pelo menos uma Visual assertion criada e testada.
- ☐ Critérios da Visual assertion revisados.
- ☐ Teste criado com Build with mabl usando Build approach Cloud.
- ☐ Outline do agente revisado antes da criação.
- ☐ Resultado de Cloud run analisado por passos e evidências.
- ☐ Você consegue diferenciar falha de produto, teste, dado e ambiente.

Referências oficiais consultadas

Documentação oficial do mabl consultada para esta atualização:

- [Configuring applications and environments](#)
- [Applications and environments](#)
- [Creating a browser test](#)
- [Assertions in the mabl Trainer](#)
- [Creating visual assertions](#)
- [Agentic test authoring for web apps](#)
- [Ad hoc test runs](#)
- [Browser test output](#)

FECHAMENTO

No mabl, low-code reduz o código necessário para automatizar o teste. A responsabilidade de QA continua sendo sua: definir o requisito, escolher o cenário, criar boas assertions, revisar o que a IA gerou e interpretar corretamente as evidências.